BBR Under Mixed Workloads: Characterizing Short-Flow Latency and a Targeted Pacing Gain Improvement

CS 534 — Computer Networks | Spring 2026 | Prof. Aruna Balasubramanian

Group Members: Mehataab Naazneen Mohammed — 117376528 | Adri Katyayan — 117353314

Proposal Due: April 6, 2026 | Final Report Due: May 8, 2026

1. Goal of the Project / Why This Is an Important Problem

BBR (Bottleneck Bandwidth and RTT) is Google's model-based TCP congestion control algorithm, now deployed at an enormous scale across YouTube, Google Search, Google Cloud, and the internal B4 WAN backbone. Industry estimates suggest BBR now handles roughly 40% of global Internet traffic on the server side. What makes BBR fundamentally different from older algorithms like CUBIC and Reno is that it does not wait for packet loss as a signal of congestion. Instead, it actively measures two physical properties of the network path: BtlBw, the bottleneck link's maximum delivery rate, and RTprop, the minimum round-trip time observed when the path has no queuing. BBR then sends exactly the Bandwidth-Delay Product (BDP = BtlBw × RTprop), keeping just enough data in flight to saturate the bottleneck without filling the buffer. The result is 2–25× higher throughput than CUBIC on high-BDP paths, and dramatically lower latency on links that CUBIC would otherwise bufferbloat.

Despite these impressive gains, BBR carries a structural flaw that becomes significant in real-world deployments. During its steady-state ProbeBW phase, BBR cycles through a 1.25× pacing gain deliberately sending 25% more data than the link can carry for one full RTT in order to probe for additional available bandwidth. This probing always builds a transient queue in the bottleneck buffer. The queue is eventually drained during the 0.75× drain phase, but it never fully empties between probing cycles; what remains is a persistent standing queue. The consequence for latency-sensitive short flows is direct and measurable: any "mice" flow (a web page load, an API call, a small RPC) that arrives while an elephant flow is mid-probe must wait behind this standing queue before its first byte is even transmitted, inflating its Flow Completion Time (FCT).

This problem matters in practice because mixed elephant/mice workloads are not an edge case they are the norm. A user streaming a video in the background while browsing the web, or a server simultaneously handling a large file upload and answering short REST API requests, is creating exactly this scenario. Yet the original BBR paper (Cardwell et al., CACM 2017) focuses entirely on bulk throughput, and the most thorough empirical evaluation to date (Cao et al., IMC 2019, Stony Brook) studies BBR and CUBIC in isolation or in competition with each other not in mixed elephant/mice workloads. This is the specific gap our project addresses. We will (1) empirically characterize how BBR elephant flows degrade mice flow FCT across a range of buffer sizes and RTTs, and (2) implement a targeted kernel-level fix by reducing the ProbeBW probe-up gain from 1.25× to values of 1.10–1.15×, measuring the FCT improvement against any resulting elephant throughput degradation. As an additional contribution, we will compare BBRv1 against Google's recently released BBRv3 to test whether the latest version already partially closes this gap.

2. Platform and Prior Experimentation

We will run all experiments on Linux + Mininet, hosted on WSL2 (Ubuntu 22.04, Linux kernel 5.15+). Mininet is a network emulator that creates real Linux network namespaces and virtual Ethernet interfaces, meaning our experiments run the actual Linux TCP stack not a simulation giving us kernel-level fidelity that a packet simulator like ns-3 cannot provide. The dumbbell topology consists of two sender hosts, one bottleneck link, and one receiver, with all link parameters (bandwidth, RTT, buffer size) controlled precisely using tc/netem for delay and tc/HTB for rate limiting. Traffic generation uses iperf3, with a congestion algorithm selected per-socket via the -C flag, allowing elephant and mice flows to use different CCAs simultaneously. The kernel modification itself targets the pacing_gain[] array in net/ipv4/tcp_bbr.c; we will compile it as a loadable kernel module using the Google BBRv3 branch (github.com/google/bbr, v3), which does not require a full kernel rebuild only the module needs to be recompiled and reloaded with insmod.

We have already completed the following setup steps: verified that both bbr and cubic are available as congestion control modules via sysctl net.ipv4.tcp_available_congestion_control; confirmed that sch_netem and sch_htb kernel modules load correctly in WSL2; ran initial iperf3 tests with BBR and CUBIC flows operating simultaneously to validate per-socket CCA control; and cloned the Mininet TCP CUBIC vs BBR reference framework

(github.com/tiuweehan/mininet-tcp-cubic-bbr) as a scaffold. Our test machine has 32 GB RAM and a 6-core i7 processor, which is more than sufficient to run Mininet topologies with concurrent flows.

3. Methodology / Approach

The project runs in three sequential phases across the six-week window before the final report deadline.

- **Phase 1 — Baseline Reproduction (Weeks 1–2):** Run BBR and CUBIC bulk flows at 100 Mbps / 40 ms RTT across buffer sizes {10 KB, 50 KB, 200 KB, 1 MB, 10 MB}. This reproduces Cao et al. Figure 2 and confirms our tc/Mininet setup produces results consistent with the published IMC 2019 findings before we extend them.
- **Phase 2 — Mixed Workload Characterization (Weeks 2–3):** Inject one continuous 120-second BBR elephant flow alongside 10 sequential 1 MB mice flows (one every 5 seconds). Sweep buffer size {10 KB, 200 KB, 10 MB} × RTT {10, 40, 100 ms} for a total of 9 parameter configurations. Primary metrics: mice FCT per flow, elephant goodput, per-flow retransmissions, queue occupancy via tc -s qdisc, and Jain's fairness index. Compare BBRv1 vs. BBRv3 as a secondary axis to assess whether the latest version improves mice FCT out of the box.
- **Phase 3 — Pacing Gain Modification (Weeks 3–4):** Modify the pacing_gain[] array in tcp_bbr.c to three probe-up variants: {1.10×, 1.15×, 1.20×}. Compile each as a separate loadable module. Re-run the full Phase 2 parameter sweep for each variant and the unmodified baseline. Quantify the FCT improvement, elephant throughput degradation, and change in retransmission rate for each gain value, producing a tradeoff curve that guides practical deployment choices.

4. Initial Hypotheses

- BBR elephant flows will cause significantly higher mice FCT than CUBIC elephant flows under shallow to medium buffers, because BBR's standing queue is persistent while CUBIC's buffer occupancy is more bursty and self-draining.
- FCT degradation will increase with RTT, because a larger RTprop means a larger BDP, which means a larger standing queue (queue size ≈ 0.25 × BDP during the 1.25× probe-up half-cycle).
- Reducing the probe-up gain to 1.10× will cut mice FCT by 15–30% relative to the 1.25× baseline, with less than 5% degradation in elephant throughput, because the peak queue built during probing scales roughly linearly with the gain excess above 1.0×.
- BBRv3 will show partial improvement in mice FCT compared to BBRv1 due to its updated pacing behavior, but will not fully eliminate the standing queue problem, leaving room for our explicit gain reduction to deliver additional gains.

5. Related Work

Cardwell et al. (CACM 2017) introduced BBR and presented its large-scale deployment results on Google's B4 backbone and YouTube CDN, establishing the throughput and latency gains that motivated its widespread adoption. Cao et al. (IMC 2019, Stony Brook University) conducted the most systematic empirical evaluation of BBR to date 640 experiments varying buffer size, RTT, and bandwidth across LAN, WAN, and Mininet environments and is the primary paper our work extends. Ware et al. (IMC 2019, CMU) developed the first analytical model of BBR competing with loss-based flows, explaining the buffer-size-dependent unfairness BBR causes. Scherrer et al. (IMC 2022, ETH Zurich) used fluid models to prove that BBR maintains a standing queue exceeding 1.5 BDP in steady state, providing the theoretical grounding for our empirical measurements. Prior work on pacing gain modification includes BBR-GC (Song et al., Sensors 2021), BBR-EFRA (Njogu et al., Computer Communications 2023), and BBR-R (Han et al., 2024), all of which modify gain parameters to address RTT fairness between competing BBR flows with different round-trip times. Our work is distinct from all of these: we target mice flow FCT in mixed elephant/mice workloads, a practically important scenario that none of the prior pacing gain papers addressed at the kernel level.